

ROBOTICS SUMMER SCHOOL - FUNDAMENTALS

Mathematics for robotics

Nonlinear Optimization

Vignesh Vembar

Morning session

Motivation: The Battery Problem

THE SCENARIO

A mobile robot logs its voltage over time. We need to predict when it hits some threshold.

THE MODEL

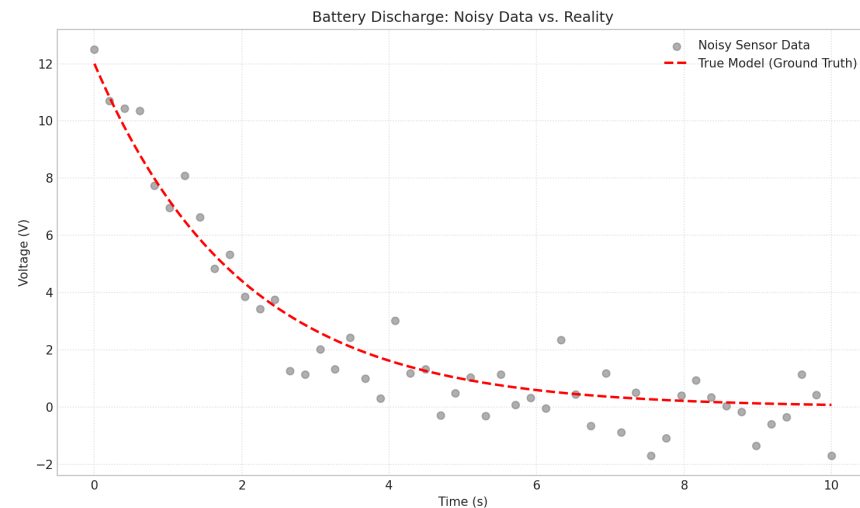
We assume an exponential discharge curve:

$$V(t) = V_0 \cdot e^{-kt}$$

THE UNKNOWNNS

We must estimate the parameters $x = [V_0, k]$ from noisy sensor data z_i at times t_i .

Since k is inside the exponent, we cannot solve for it using basic algebra. We need **Optimization**.



Linear vs. Nonlinear

Linear Systems

Equation: $Ax = b$

Battery Analogy: If the battery died linearly: $V(t) = V_0 - kt$.

Solution: Direct matrix inversion.

$$x = A^{-1}b$$

Result: Instant, unique solution.

Nonlinear Systems

Equation: $f(x) = 0$

Battery Analogy: Real physics: $V(t) = V_0 e^{-kt}$.

Solution: Iterative approximation. We guess x_k , calculate error, adjust x .

Result: Slow, might get stuck in local minima.

The Jacobian Matrix (J)

Definition: Sensitivity of the Prediction

The Jacobian tells us how a tiny change in our *control variables* (V_0, k) affects the *prediction* at *every single time step* ($t_1, t_2, \dots, t_N$).

Structure

A Matrix of size $N \times 2$ (Rows = Data Points, Cols = Parameters).

It stacks the partial derivatives of the model $V(t)$ for every time step:

$$J = \begin{pmatrix} \frac{\partial V(t_1)}{\partial V_0} & \frac{\partial V(t_1)}{\partial k} \\ \dots & \dots \\ \frac{\partial V(t_N)}{\partial V_0} & \frac{\partial V(t_N)}{\partial k} \end{pmatrix}$$

For the battery model $V(t) = V_0 e^{-kt}$:

$$J = \begin{pmatrix} -e^{-kt_1} & -V_0 t_1 e^{-kt_1} \\ \dots & \dots \\ -e^{-kt_N} & -V_0 t_N e^{-kt_N} \end{pmatrix}$$

Why is it useful?

It creates a linear map from “Parameter Space” to “Sensor Space”.

- **Local Linearization:** If we change parameters by a tiny vector δ , the sensor readings change by $J\delta$.

The Gradient Vector (∇C)

Definition: Slope of the Total Cost

The Gradient is a vector representing how the *total sum of squared errors* (C) changes with respect to each control variable.

Structure

A Vector of size 2×1 (One component per parameter).

It aggregates the errors from *all data points* into a single direction:

$$\nabla C = \begin{pmatrix} \frac{\partial C}{\partial V_0} \\ \frac{\partial C}{\partial k} \end{pmatrix}$$

Using the Chain Rule ($C = \sum r_i^2$), we get:

$$\frac{\partial C}{\partial V_0} = -2 \sum r_i e^{-kt_i}$$

$$\frac{\partial C}{\partial k} = 2 \sum r_i V_0 t_i e^{-kt_i}$$

Why is it useful?

It points in the direction of the **steepest ascent**.

The Hessian Matrix (H)

Definition: Curvature of the Cost Landscape

The Hessian describes how the *slope itself changes*. It tells us if the valley is narrow and steep, or wide and flat.

Structure

A Square Matrix of size 2×2 .

It contains second derivatives (cross terms):

$$H = \begin{pmatrix} \frac{\partial^2 C}{\partial V_0^2} & \frac{\partial^2 C}{\partial V_0 \partial k} \\ \frac{\partial^2 C}{\partial k \partial V_0} & \frac{\partial^2 C}{\partial k^2} \end{pmatrix}$$

- **Note:** It is symmetric. The mixed term (top-right) is the same as bottom-left.

Why is it useful?

It captures the *geometry* of the quadratic bowl.

- **Second-Order Methods:** Newton and Levenberg-Marquardt use H to calculate a jump straight to the minimum, rather than walking down the slope.
- **The “Eigenvalues”:** If the eigenvalues of H are large, the valley is narrow (easy to converge). If they are small, the valley is flat (hard to converge).

The Cost Function

How do we measure “how good” our parameters $[V_0, k]$ are?

We define a **Cost Function** $C(x)$ (also called Objective or Loss). For our battery problem, we use the Sum of Squared Errors:

$$C(V_0, k) = \sum_{i=1}^N (z_i - V_0 e^{-kt_i})^2$$

Our Goal

Find the specific V_0 and k that make this value as small as possible (ideally zero).

The Challenge

This equation is non-convex and complex. There is no formula to invert it. We must search.

The Residual Vector $r(x)$

The difference between the measurement z_i and the model prediction $h(t_i)$:

$$r_{i(x)} = z_i - V_0 e^{-kt_i}$$

Constraints & Lagrange

Sometimes physics limits our parameters. For the battery, the decay rate k must be positive ($k > 0$). A battery cannot charge itself spontaneously.

We turn a **Constrained** problem into an **Unconstrained** problem.

The Lagrangian

$$\mathcal{L}(x, \lambda) = C(x) - \lambda g(x)$$

Where:

- $C(x)$: Our battery cost function.
- $g(x)$: The constraint function (e.g., $k - \varepsilon = 0$).
- λ : The “force” required to keep us on the constraint boundary.

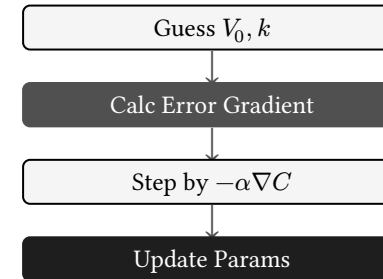
Gradient Descent

The simplest way to solve for $[V_0, k]$: Take small steps downhill.

Algorithm

1. Guess initial $[V_0, k]$.
2. Calculate slope ∇C .
3. Update: $x_{k+1} = x_k - \eta \nabla C(x_k)$
4. Repeat until slope is flat.

η = Step size (Learning Rate).

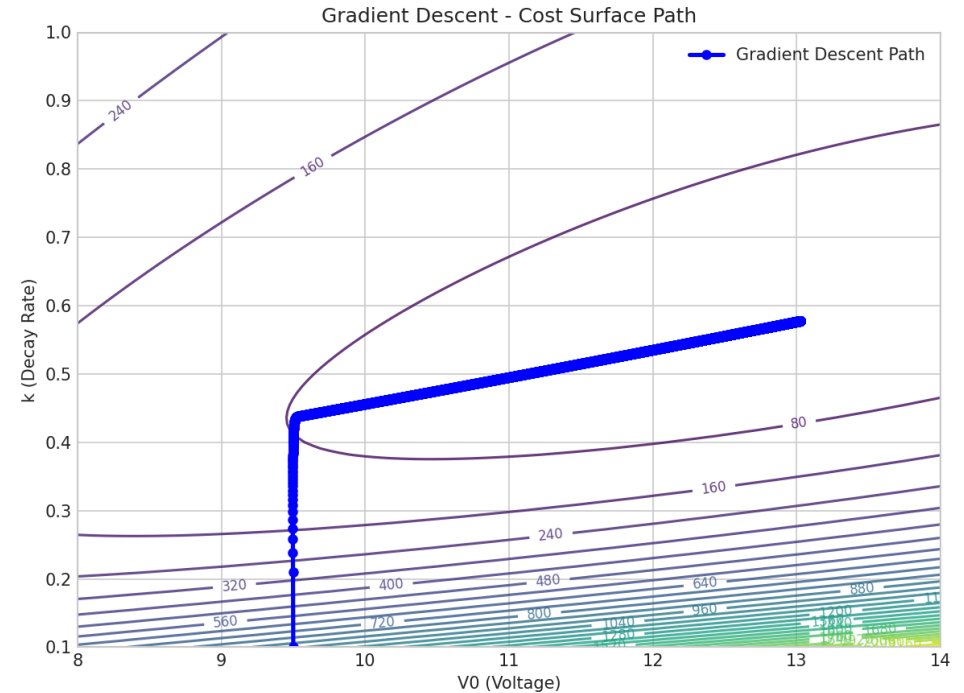


Gradient Descent: Code & Visualization

Pseudocode

```
x = x0          # Initial guess
alpha = 0.00001 # Step size (1e-5)
tol = 1e-4      # Early stopping tolerance
prev_cost = C(x)

for i in range(100000):
    grad = nabla C(x) # Gradient
    x = x - alpha * grad
    if abs(C(x) - prev_cost) < tol:
        break # Converged!
    prev_cost = C(x)
```



Adam Optimizer: Code & Visualization

Pseudocode

```
m = 0, v = 0          # Moments
lr = 0.1
tol = 1e-4
prev_cost = C(x)

for i in range(1000):
    grad = nabla C(x)

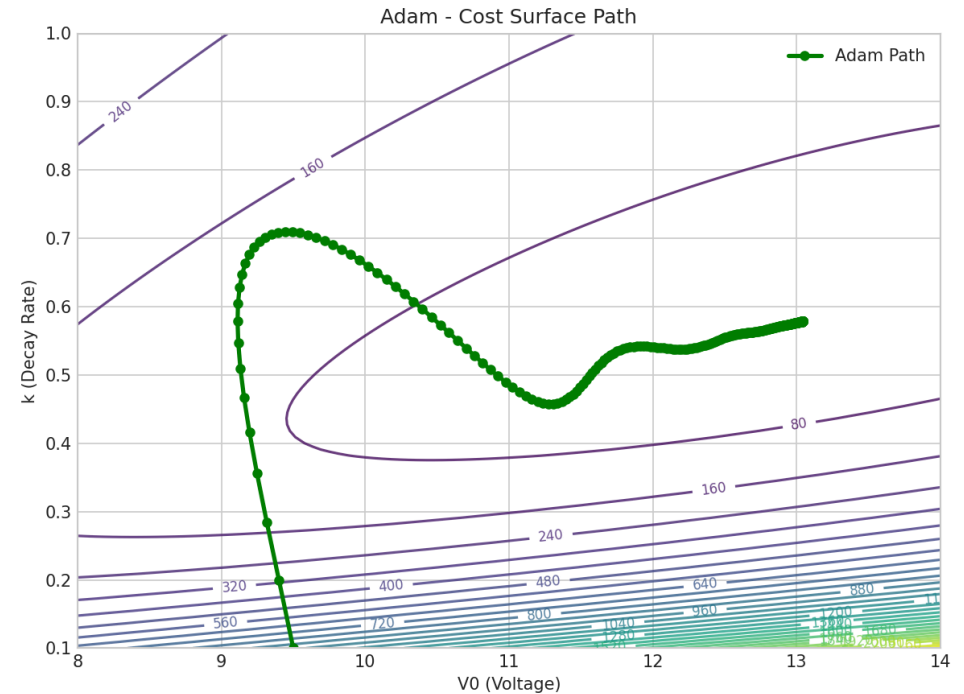
    m = 0.9*m + 0.1*grad    # Momentum
    v = 0.999*v + 0.001*grad^2

    m_hat = m / (1-0.9^i)
    v_hat = v / (1-0.999^i)

    x = x - lr * m_hat / (sqrt(v_hat) + eps)

    if abs(C(x) - prev_cost) < tol:
        break # Converged!
    prev_cost = C(x)
```

Key: Momentum + Adaptive learning rates



Newton's Method: Second-Order Derivation

Newton's method does not just look at the slope; it looks at the **curvature** of the valley.

1. Quadratic Approximation

We approximate the Cost Function $C(x)$ using a 2nd-Order Taylor Series expansion around our current point x :

$$C(x + \delta) \approx C(x) + \nabla C(x)^T \delta + \frac{1}{2} \delta^T H \delta$$

2. Find the Minimum of the Bowl

We find the step δ that makes the gradient of this quadratic approximation zero.

$$\nabla_{\delta} \left(C(x) + \nabla C^T \delta + \frac{1}{2} \delta^T H \delta \right) = 0$$

$$\nabla C + H \delta = 0$$

Solving for δ :

$$\delta = -H^{-1} \nabla C$$

Result: Newton can “jump” directly to the bottom of the bowl.

The Hessian Challenge

If Newton is so good, why don't we use it everywhere? The **Hessian** H is expensive to compute. Inverting the Hessian matrix is $O(N^3)$. For a system with more degrees of control, this is too slow for real-time control.

The Gauss-Newton Idea

Computing the full Hessian is messy. But notice that near the minimum, $r_i \approx 0$.

If we ignore the second-order error terms, can we approximate H using only the Jacobian J (which we already have)?

The Shortcut

Instead of computing complex 2nd derivatives, we use:

$$H \approx J^T J$$

This gives us quadratic convergence **without** the calculus headache!

Gauss-Newton: The Derivation

Newton's method is too expensive because we need the Hessian H . However, for Least Squares, we can approximate H using the Jacobian J .

1. Taylor Expansion of Residuals

We approximate the residual vector $r(x)$ at the new point $x + \delta$:

$$r(x + \delta) \approx r(x) + J(x)\delta$$

2. Approximate the Cost Function

$$\begin{aligned} C(x + \delta) &\approx \| r + J\delta \|^2 = (r + J\delta)^T (r + J\delta) \\ &= r^T r + r^T J\delta + \delta^T J^T r + \delta^T J^T J\delta \end{aligned}$$

Since $r^T J\delta$ is a scalar (equals its transpose):

$$\approx C(x) + 2\delta^T J^T r + \delta^T (J^T J)\delta$$

3. Minimize w.r.t δ

Take gradient of quadratic approximation and set to zero:

$$2J^T r + 2J^T J\delta = 0$$

$$(J^T J)\delta = -J^T r$$

Gauss-Newton: Code & Visualization

Pseudocode

```
tol = 1e-4
prev_cost = C(x)

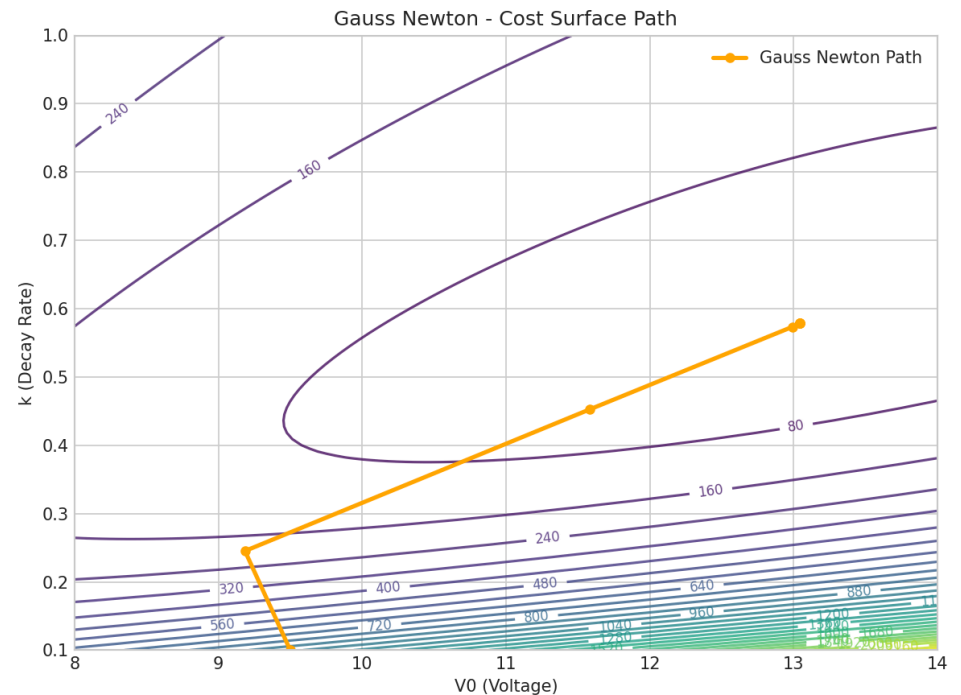
for i in range(100): # Max iterations
    r = z - h(x)      # Residuals
    J = dr/dx         # Jacobian

    # Gauss-Newton update
    delta = (JT J)-1 * (-JT r)
    x = x + delta

    if abs(C(x) - prev_cost) < tol:
        break # Converged!
    prev_cost = C(x)
```

Approximates Hessian as $J^T J$

Fast convergence near minimum. Fails if J is singular.



Levenberg-Marquardt: The Derivation

What if $J^T J$ is singular? We add a “Damping” term λI derived from a Trust Region.

Trust Region Formulation

We want to find a step δ that minimizes the quadratic approximation, BUT we also restrict the step size to a radius Δ (Trust Region):

$$\min_{\delta} \|J\delta + r\|^2 \quad \text{s.t.} \quad \|\delta\|^2 \leq \Delta^2$$

Lagrangian

Solving this constrained problem introduces a Lagrange multiplier λ (related to the trust region radius):

$$(J^T J + \lambda I)\delta = -J^T r$$

High λ

λI

dominates diagonal. Behaves like **Gradient Descent**.

Small, stable steps. Far from min.

Low λ

$J^T J$

dominates. Behaves like **Gauss-Newton**.

Fast, quadratic convergence. Near min.

Levenberg-Marquardt: Code & Visualization

Pseudocode

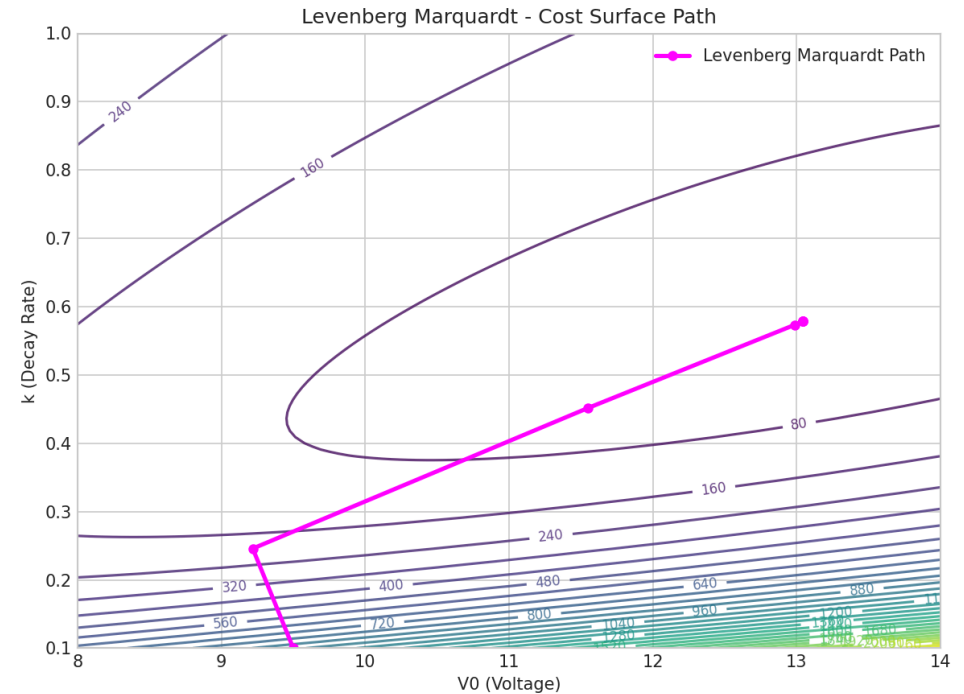
```
lambda = 1.0
tol = 1e-4
prev_cost = C(x)

for i in range(100): # Max iterations
    r = z - h(x)
    J = dr/dx

    # LM update (Trust Region)
    delta = (J^T J + lambda*I)^-1 * (-J^T r)

    if cost(x + delta) < cost(x):
        x = x + delta
        lambda /= 10 # Trust region grew
    else:
        lambda *= 10 # Shrink step

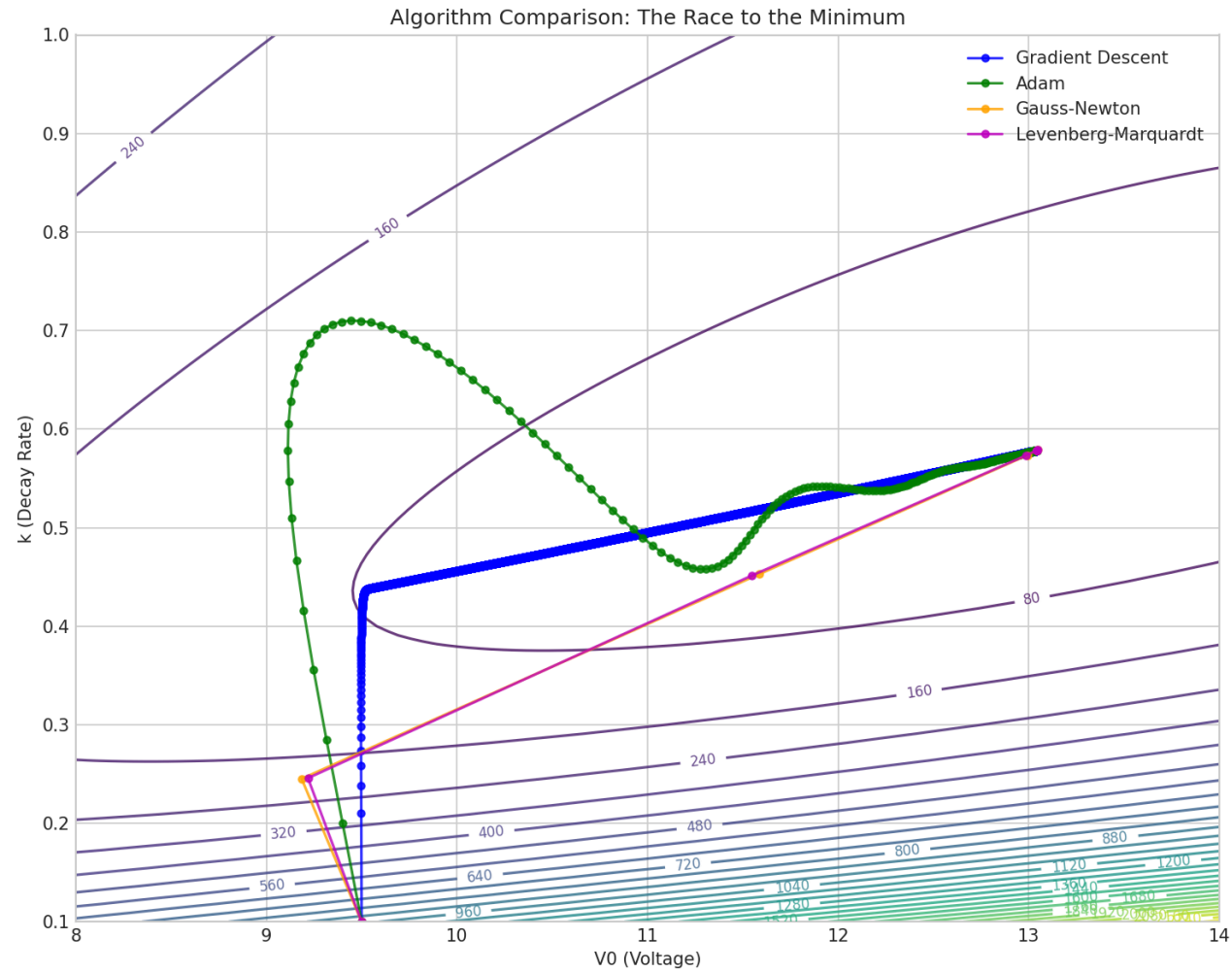
    if abs(C(x) - prev_cost) < tol:
        break # Converged!
    prev_cost = C(x)
```



Experimental Results

How do they actually perform on the battery data? (Noise $\sigma = 1.0$)

Method	Iterations	Time (s)	Cost
Gradient Descent	$97,018 \pm 4,435$	1.47 ± 0.09	47.44 ± 8.75
Adam	254 ± 33	0.0055 ± 0.0007	47.44 ± 8.75
Gauss-Newton	7 ± 1	0.0002 ± 0.0000	47.44 ± 8.75
Levenberg-Marquardt	7 ± 1	0.0002 ± 0.0000	47.44 ± 8.75



Conclusion: Algorithm Selection Guide

Ask Yourself:

1. Is your problem **Linear**? (e.g., $Ax = b$, Triangulation)
 - If Yes: Use **SVD / Matrix Inversion**.
 - It is instant and exact.
2. Is it **Deep Learning**? (e.g., CNNs, LLMs, Stochastic Data)
 - If Yes: Use **Adam / SGD**.
 - Hessian is too big ($O(N^2)$). Mini-batches break curvature.
3. Is it **SLAM / Curve Fitting**? (e.g., Battery, BA, Optimization)
 - If Yes: Use **Gauss-Newton / LM**.
 - Least Squares structure allows fast, quadratic convergence.

The Verdict

For the battery problem and most State Estimation tasks, **Levenberg-Marquardt** is the winner.

It combines the stability of Gradient Descent (start) with the speed of Gauss-Newton (finish).

Don't reinvent the wheel. Use `scipy.optimize.least_squares`.

Questions?